

MANUAL TÉCNICO

Sistema de estacionamiento

1. Introducción

En este proyecto desarrollé un sistema de estacionamiento que funciona desde consola. El programa permite ingresar y retirar vehículos, mostrar el estado del parqueo, buscar un vehículo por medio de su placa, calcular la ruta más corta entre la entrada y la salida y llevar el control de los ingresos obtenidos por los pagos.

El estacionamiento se representa con una matriz de 10 por 10 posiciones. El borde exterior se utiliza como vía de circulación y contiene una entrada y una salida. Las 64 posiciones internas representan los espacios disponibles para estacionar vehículos.

2. Objetivo

El objetivo fue aplicar los conceptos básicos de programación orientada a objetos, matrices, arreglos, ciclos, métodos, validaciones y estructuras de control. También se buscó mantener una solución sencilla y entendible, sin utilizar colecciones o estructuras dinámicas.

3. Herramientas

Lenguaje y versión: Java Development Kit 25.

Entorno de desarrollo: Apache NetBeans 28.

4. Organización general del programa

El programa se dividió en cuatro clases para que cada parte tuviera una responsabilidad clara.

Estacionamiento: Contiene el método main. En este método se crean los objetos Tablero y Menu, se inicializa el estacionamiento, se colocan la entrada y la salida y finalmente se muestra el menú.

Menu: Muestra las opciones disponibles, lee la elección del usuario y llama al método correspondiente de la clase Tablero.

Tablero: Guarda el estado del parqueo y contiene la mayor parte de la lógica: ingreso, retiro, búsqueda, pago, conteo de espacios, ingresos y cálculo de la ruta.

Validaciones: Contiene las comprobaciones relacionadas con el formato de la placa y la búsqueda de placas repetidas.

5. Arreglos y variables utilizadas

5.1 Matriz tablero

La variable tablero es una matriz de String con tamaño 10 x 10. En ella se guarda la parte visual y lógica del estacionamiento. Se eligió String porque cada posición contiene un símbolo que identifica su estado.

```
private String[][] tablero = new String[10][10];
```

Símbolo =: Representa la vía exterior.

Símbolo L: Representa un espacio interno libre.

Símbolo A: Representa un automóvil estacionado.

Símbolo E: Representa la entrada colocada sobre el borde.

Símbolo S: Representa la salida colocada sobre el borde.

5.2 Matriz placas

La matriz placas también tiene un tamaño de 10 x 10. Su posición corresponde directamente con la posición de la matriz tablero. Cuando hay un automóvil, en placas se guarda su número de placa; cuando no hay vehículo, la posición queda con el valor null.

```
private String[][] placas = new String[10][10];
```

Utilicé una segunda matriz porque el tablero solamente necesita mostrar símbolos. De esta forma puedo buscar la placa sin cambiar la representación visual del estacionamiento.

5.3 Arreglo de posición

El método buscarVehiculo devuelve un arreglo de dos enteros. La posición cero guarda la fila y la posición uno guarda la columna. Si no se encuentra el vehículo, el método devuelve null.

```
int[] posicion = new int[2];
```

5.4 Variables de control

ingresos: Acumula únicamente la tarifa cobrada de Q10.00 por cada ingreso válido.

contadorVehiculos: Cuenta cuántos vehículos han realizado el pago.

filaEntrada, columnaEntrada, filaSalida y columnaSalida: Guardan las coordenadas generadas para la entrada y la salida. Estas variables se usan al calcular las rutas del perímetro.

6. Descripción de los métodos

6.1 llenarTablero

Recorre las 100 posiciones de la matriz mediante dos ciclos for. Si la posición pertenece a la primera o última fila, o a la primera o última columna, se coloca el símbolo =. En las demás posiciones se coloca L. De esta manera se forman el borde exterior y los 64 espacios internos.

6.2 mostrarTablero

Imprime los números de las filas y columnas internas y después muestra cada elemento de la matriz. Mientras recorre los espacios internos también cuenta cuántos contienen L y cuántos contienen A. Al final informa la cantidad de espacios libres y ocupados.

6.3 colocarEntradaSalida

Llama dos veces al método colocarEnBorde: primero con la letra E y después con la letra S. La llamada se hace una sola vez durante la inicialización del programa para evitar que las posiciones cambien cada vez que se muestra el menú.

6.4 colocarEnBorde

Genera una fila y una columna aleatorias entre 0 y 9. La posición solamente se acepta si contiene el símbolo = y no es una esquina. Cuando se coloca E o S también se guardan sus coordenadas. Como la entrada ocupa el lugar seleccionado, la salida no puede colocarse encima de ella.

6.5 ingresarVehiculo

Primero comprueba si el estacionamiento está lleno. Después solicita la placa, valida su formato y verifica que no esté repetida. Luego pide la fila y la columna, comprueba que estén dentro del rango de 1 a 8 y confirma que la posición esté libre. Solamente después de estas comprobaciones se realiza el pago. Al finalizar se cambia L por A y se guarda la placa en la misma posición de la segunda matriz.

6.6 realizarPago

Solicita el monto entregado y repite la pregunta mientras sea menor que Q10.00. Un monto negativo también es rechazado. Cuando el pago es suficiente se calcula el cambio con monto - 10, se suman Q10.00 a los ingresos y se incrementa el contador de vehículos cobrados.

6.7 mostrarIngresos

Muestra el número acumulado de vehículos cobrados y el total de ingresos. Los ingresos aumentan con la tarifa, no con la cantidad completa que entrega el usuario, porque la diferencia se devuelve como cambio.

6.8 buscarVehiculo

Solicita una placa y valida su formato. Después recorre las filas y columnas internas de la matriz placas. Para evitar un error, primero comprueba que la posición sea diferente de null y luego usa equals para comparar. Si encuentra una coincidencia, muestra la fila y columna y

devuelve un arreglo con esas coordenadas. Si termina el recorrido sin encontrarla, muestra un mensaje y devuelve null.

6.9 retirarVehiculo

Reutiliza el método buscarVehiculo para no repetir el recorrido de la matriz. Si la posición recibida es diferente de null, cambia el espacio del tablero a L y asigna null en la posición correspondiente de placas. Con esto el espacio vuelve a quedar disponible.

6.10 estacionamientoLleno

Recorre las 64 posiciones internas. Si encuentra por lo menos una L devuelve false inmediatamente. Si termina todos los ciclos sin encontrar espacios libres, devuelve true y el ingreso de un nuevo vehículo se detiene.

7. Validaciones

7.1 Validación del formato de la placa

El formato aceptado es P###LLL. Esto significa que la placa debe tener exactamente siete caracteres: la letra P mayúscula, tres números y tres letras mayúsculas. Un ejemplo válido es P123ABC.

Primero se verifica la longitud con length(). Después se revisa el primer carácter con charAt(0). Las posiciones 1, 2 y 3 se comprueban con Character.isDigit(). Las posiciones 4, 5 y 6 se comprueban con Character.isLetter() y Character.isUpperCase(). Si alguna condición no se cumple, el método muestra el mensaje correspondiente y devuelve false.

7.2 Placa repetida

Antes de guardar un vehículo se recorre la matriz placas. Cada posición se compara únicamente si no es null. Si se encuentra la misma placa, se informa que ya está registrada y se cancela el ingreso.

7.3 Fila y columna

Los vehículos solamente pueden estacionarse en las posiciones internas, por eso la fila y la columna deben estar entre 1 y 8. Además, la posición elegida debe contener L. Estas comprobaciones se realizan antes de cobrar para evitar registrar un pago cuando el espacio no puede utilizarse.

7.4 Pago

La tarifa es Q10.00. El ciclo do-while solicita nuevamente el monto cuando es negativo o menor que la tarifa. El vehículo solo se registra cuando el pago ya fue aceptado.

7.5 Estacionamiento lleno

Antes de pedir los datos del vehículo se revisan los 64 espacios internos. Si todos contienen A, el sistema muestra que el estacionamiento está lleno y regresa al menú.

7.6 Opción del menú

Antes de leer la opción con `nextInt()` se utiliza `hasNextInt()`. Si el usuario escribe letras u otro dato que no sea entero, el valor se descarta con `next()` y el menú vuelve a mostrarse. Si se ingresa un número fuera del rango de 1 a 7, se muestra el mensaje de opción incorrecta.

7.7 Búsqueda y retiro

La búsqueda valida la placa antes de recorrer la matriz. El retiro solamente modifica el tablero cuando `buscarVehiculo` devuelve una posición válida. Si el resultado es null, no se cambia ningún dato.

8. Cálculo de la ruta más corta

8.1 Idea utilizada

La ruta se calcula únicamente entre la entrada y la salida por el borde exterior. No se calcula desde la posición de un automóvil. Para simplificar el problema convertí cada coordenada del borde en una sola posición numérica, siguiendo el sentido horario.

El perímetro tiene 36 posiciones diferentes. La fila superior aporta 10 posiciones, el lado derecho aporta 8 sin repetir las esquinas, la fila inferior aporta 10 y el lado izquierdo aporta 8. Por eso el total es $10 + 8 + 10 + 8 = 36$.

8.2 Conversión de coordenadas

El método `obtenerPosicionBorde` asigna un número de 0 a 35 según el lado en el que se encuentre la coordenada. El recorrido comienza en la esquina superior izquierda y avanza en sentido horario.

Fila superior, fila = 0: posición = columna

Lado derecho, columna = 9: posición = 9 + fila

Fila inferior, fila = 9: posición = 27 - columna

Lado izquierdo, columna = 0: posición = 36 - fila

Estas expresiones hacen que las posiciones aumenten de manera continua alrededor de todo el borde. Aunque las esquinas forman parte del perímetro, la entrada y la salida nunca se colocan en ellas.

8.3 Distancia en sentido horario

Si la posición de salida es mayor o igual que la posición de entrada, la distancia se obtiene restando salida - entrada. Si la salida aparece antes que la entrada en la numeración, primero se recorre lo que falta hasta completar las 36 posiciones y después se suma la posición de salida.

```
rutaHoraria = posicionSalida - posicionEntrada  
rutaHoraria = 36 - posicionEntrada + posicionSalida
```

8.4 Distancia en sentido antihorario

Los dos recorridos juntos completan el perímetro. Por esta razón, después de obtener la ruta horaria, la distancia antihoraria se calcula restando ese resultado a 36.

$$\text{rutaAntihoraria} = 36 - \text{rutaHoraria}$$

8.5 Comparación

Finalmente se comparan las dos distancias. Si la horaria es menor, se recomienda el sentido horario. Si la antihoraria es menor, se recomienda ese sentido. Cuando ambas son iguales, el programa indica que cualquiera de las dos rutas puede utilizarse.

8.6 Ejemplo

Si la entrada aparece en la fila 1, columna 4, internamente corresponde a [0][3] y su posición de borde es 3. Si la salida aparece en la fila 7, columna 10, corresponde a [6][9] y su posición es 15. La ruta horaria mide $15 - 3 = 12$ posiciones. La ruta antihoraria mide $36 - 12 = 24$ posiciones. Por lo tanto, la ruta recomendada es la de sentido horario.

9. Decisiones de implementación

9.1 Uso de matrices

Elegí dos matrices de tamaño fijo porque el estacionamiento tiene dimensiones conocidas y porque no se permiten estructuras dinámicas. La relación entre tablero y placas se mantiene usando los mismos índices de fila y columna.

9.2 Separación de responsabilidades

Separé el inicio del programa, el menú, la lógica del tablero y las validaciones. Esto facilita leer el código y permite modificar una parte sin mezclarla con todas las demás. La clase Tablero conserva los datos porque casi todas las operaciones necesitan consultar o cambiar el estacionamiento.

9.3 Reutilización de la búsqueda

El retiro llama a buscarVehiculo en lugar de repetir el mismo recorrido. La búsqueda devuelve la posición encontrada y el retiro se limita a liberarla. Esto reduce código repetido y mantiene una sola forma de localizar placas.

9.4 Entrada y salida aleatorias

La entrada y la salida se generan al iniciar el programa. Se excluyen las esquinas para que queden sobre un lado definido del perímetro. También se evita que ocupen la misma posición porque la segunda solo puede colocarse sobre un símbolo =.

9.5 Control de ingresos

El sistema suma Q10.00 por cada pago aprobado. El contador de vehículos cobrados no disminuye al retirar un automóvil, porque representa la cantidad de cobros realizados durante la ejecución. En cambio, la cantidad actual de espacios ocupados se obtiene recorriendo el tablero.